

Lab- 01: Introduction to Machine Learning and Python Environment

Aim: To get familiar with machine learning concepts and set up the Python environment for ML experiments.

```
import numpy as np, pandas as pd, matplotlib.pyplot as plt
import sklearn
print(np.__version__, pd.__version__, sklearn.__version__)
```

```
2.0.2 2.2.2 1.6.1
```

```
# experiment no 2: basic python & numpy
a = [1,2,3]
d = {"x":1}
for i in a: print(i)
import numpy as np
arr = np.array([1,2,3])
print(arr.mean(), arr.sum())
```

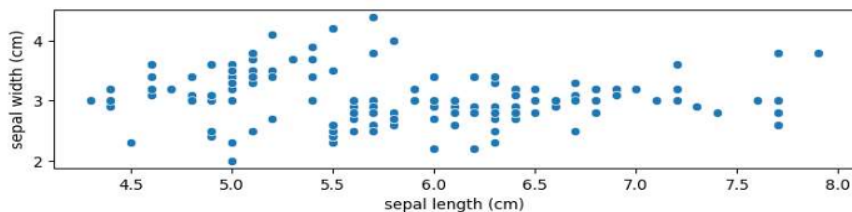
```
1
2
3
2.0 6
```

```
# experiment no 3: load & explore dataset
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
print(df.head(2))
print(df.describe())
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
0	5.1	3.5	1.4	0.2
1	4.9	3.0	1.4	0.2

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

```
# experiment no 4: data visualization
import seaborn as sns
import matplotlib.pyplot as plt
plt.figure(figsize=(9, 2))
sns.scatterplot(data=df, x=df.columns[0], y=df.columns[1])
plt.show()
```



```
# experiment no 5: train-test split
from sklearn.model_selection import train_test_split
X = df
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
print(X_train.shape, X_test.shape)
```

```
(120, 4) (30, 4)
```

Result The Python environment was successfully set up and basic ML workflow was performed.

Lab– 02: Data Preprocessing in Machine Learning

Aim: To understand and implement techniques for cleaning, transforming, and preparing data for machine learning.

```
# experiment no 1: handling missing values
import pandas as pd
df = pd.DataFrame({
    "age": [20, None, 22],
    "city": ["dhaka", "ctg", None]
})
print(df.isnull().sum())
df["age"] = df["age"].fillna(df["age"].mean())
df["city"] = df["city"].fillna(df["city"].mode()[0])
print(df)
```

```
age    1
city    1
dtype: int64
   age  city
0  20.0 dhaka
1  21.0   ctg
2  22.0   ctg
```

```
# experiment no 2: encoding categorical data
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["city_label"] = le.fit_transform(df["city"])
df_onehot = pd.get_dummies(df["city"], prefix="city")
print(df_onehot)
```

```
   city_ctg  city_dhaka
0     False         True
1      True         False
2      True         False
```

Result:

False True True False True True False False Data preprocessing techniques including handling missing values and encoding categorical data were successfully implemented. The dataset is now suitable for machine learning models.

Lab-03: Exploratory Data Analysis (EDA) in Machine Learning

Aim: To explore and analyze datasets to uncover patterns, detect anomalies, and summarize key statistics.

```
# experiment no 1: loading and inspecting data
import pandas as pd
from sklearn.datasets import load_iris
iris = load_iris(); df = pd.DataFrame(iris.data, columns=iris.feature_names)
print("Shape:", df.shape); print(df.head(2))
print(df.tail(2)); print(df.describe().loc[['mean', '50%', 'max']])
```

```
Shape: (150, 4)
   sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                5.1                3.5                1.4                0.2
1                4.9                3.0                1.4                0.2
148               6.2                3.4                5.4                2.3
149               5.9                3.0                5.1                1.8
mean             5.843333             3.057333             3.758             1.199333
50%              5.800000             3.000000             4.350             1.300000
max              7.900000             4.400000             6.900             2.500000
```

```
# experiment no 2: statistical summary of data
import pandas as pd
stats = pd.DataFrame({
    'mean': df.mean().round(2),
    'median': df.median().round(2),
    'std': df.std().round(2),
    'var': df.var().round(2)
})
print(stats); print("\nCorrelation:\n", df.corr().round(2))
```

```
   sepal length (cm)  mean  median  std  var
sepal length (cm)  5.84   5.80  0.83  0.69
```

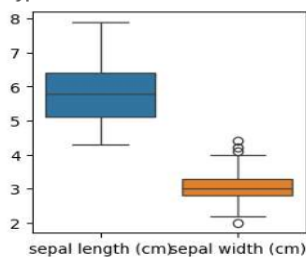
```
sepal width (cm)  3.06   3.00  0.44  0.19
petal length (cm)  3.76   4.35  1.77  3.12
petal width (cm)  1.20   1.30  0.76  0.58
target           1.00   1.00  0.82  0.67
```

```
Correlation:
   sepal length (cm)  sepal width (cm)  petal length (cm)  \
sepal length (cm)         1.00         -0.12         0.87
sepal width (cm)         -0.12         1.00         -0.43
petal length (cm)         0.87         -0.43         1.00
petal width (cm)         0.82         -0.37         0.96
target                   0.78         -0.43         0.95

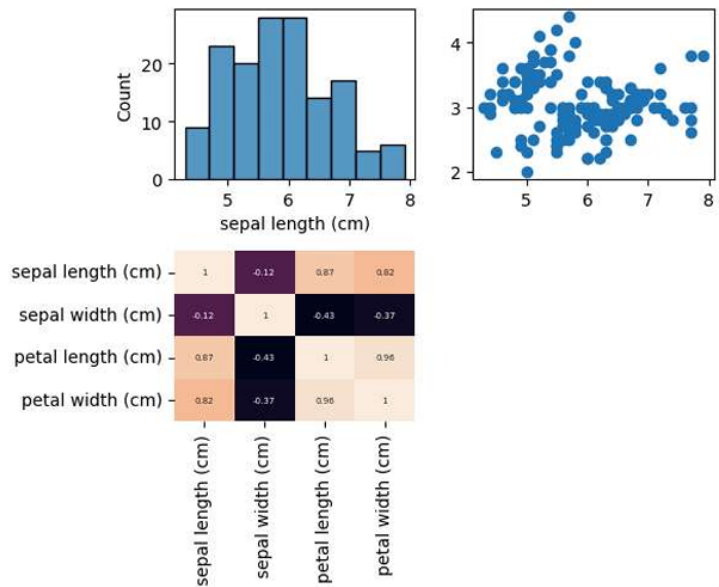
   petal width (cm)  target
sepal length (cm)   0.82   0.78
sepal width (cm)   -0.37  -0.43
petal length (cm)   0.96   0.95
petal width (cm)   1.00   0.96
target              0.96   1.00
```

```
# experiment no 3: missing values and outliers
print(df.isnull().sum())
import seaborn as sns; import matplotlib.pyplot as plt
plt.figure(figsize=(3,3)); plt.tight_layout()
sns.boxplot(data=df.iloc[:,0:2]); plt.show()
```

```
sepal length (cm)  0
sepal width (cm)  0
petal length (cm)  0
petal width (cm)  0
dtype: int64
```

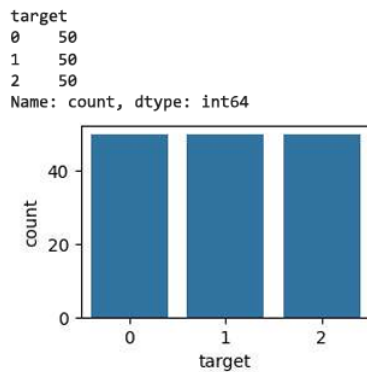


```
# experiment no 4: data visualization
import seaborn as sns;import matplotlib.pyplot as plt
plt.figure(figsize=(6,5));plt.subplot(2,2,1);sns.histplot(df.iloc[:,0]);plt.subplot(2,2,2);plt.scatter(df.iloc[:,0], df.iloc[:,1]);plt.subplot(2,2,3);sns.heatmap(df.corr(), annot=True, annot_kws={'size':5}, cbar=False);plt.tight_layout();plt.show()
```



```
# experiment no 5: feature analysis
df['target'] = iris.target
print(df['target'].value_counts())
import seaborn as sns

import matplotlib.pyplot as plt
plt.figure(figsize=(3,2))
sns.countplot(x='target', data=df)
plt.show()
```



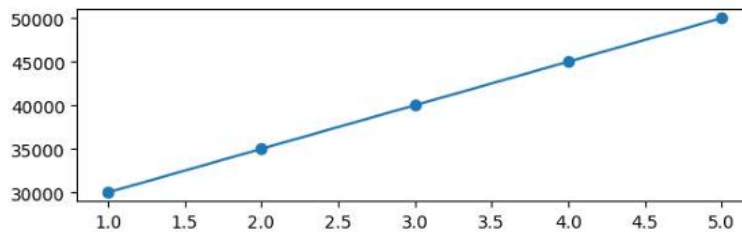
Result:

Exploratory Data Analysis was successfully performed. Statistical summaries, visualizations, and feature analysis helped in understanding data distribution, relationships, and potential anomalies.

Lab– 04: Linear Regression in Machine Learning

Aim: To implement and evaluate simple and multiple linear regression models using Python.

```
# experiment no 1: simple linear regression
from sklearn.linear_model import LinearRegression
import matplotlib.pyplot as plt
X=[[1],[2],[3],[4],[5]];y=[30000,35000,40000,45000,50000]
m=LinearRegression();m.fit(X,y)
yp=m.predict(X);plt.figure(figsize=(7,2));plt.scatter(X,y);plt.plot(X,yp);plt.show()
```



```
# experiment no 2: multiple linear regression
from sklearn.datasets import load_diabetes
d=load_diabetes();X=d.data;y=d.target
m=LinearRegression();m.fit(X,y)
```

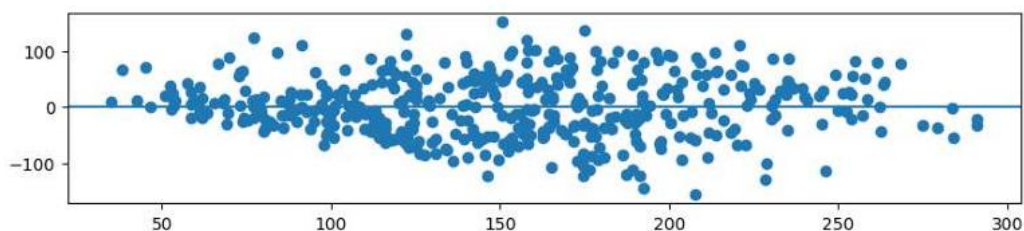
LinearRegression [?](#)

LinearRegression()

```
# experiment no 3: model evaluation
from sklearn.metrics import mean_absolute_error,mean_squared_error,r2_score
import numpy as np
yp=m.predict(X)
print(mean_absolute_error(y,yp),mean_squared_error(y,yp),np.sqrt(mean_squared_error(y,yp)),r2_score(y,yp))
```

43.27745202531506 2859.69634758675 53.47612876402657 0.5177484222203499

```
# experiment no 4: residual visualization
r=y-yp;plt.figure(figsize=(10,2));plt.scatter(yp,r);plt.axhline(0);plt.show()
```



```
# experiment no 5: predicting new data
print(m.predict([X[0]]))
```

[206.11667725]

Result

Simple and multiple linear regression models were successfully implemented. Model performance was evaluated using standard regression metrics, and residual analysis confirmed the validity of model assumptions.

Lab- 05: Logistic Regression in Machine Learning

Aim: To implement and evaluate logistic regression models for classification tasks using Python

```
# experiment no 1: binary logistic regression
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
d=load_iris();X=d.data;y=(d.target==0).astype(int)
Xt,Xe,yt,ye=train_test_split(X,y,test_size=0.2)
m=LogisticRegression(max_iter=200);m.fit(Xt,yt)
yp=m.predict(Xe);print(accuracy_score(ye,yp))
```

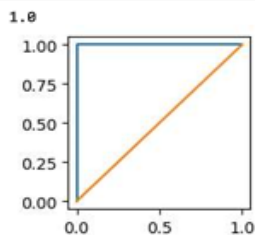
1.0

```
# experiment no 2: confusion matrix & classification report
from sklearn.metrics import confusion_matrix,classification_report
print(confusion_matrix(ye,yp))
print(classification_report(ye,yp))
```

```
[[22  0]
 [ 0  8]]
```

	precision	recall	f1-score	support
0	1.00	1.00	1.00	22
1	1.00	1.00	1.00	8
accuracy			1.00	30
macro avg	1.00	1.00	1.00	30
weighted avg	1.00	1.00	1.00	30

```
# experiment no 3: roc curve & auc
from sklearn.metrics import roc_curve,auc
import matplotlib.pyplot as plt
ys=m.predict_proba(Xe)[:,-1]
fpr,tpr,_=roc_curve(ye,ys);print(auc(fpr,tpr))
plt.figure(figsize=(2,2));plt.plot(fpr,tpr);plt.plot([0,1],[0,1]);plt.show()
```



```
# experiment no 4: multiclass logistic regression
d=load_iris();X=d.data;y=d.target
Xt,Xe,yt,ye=train_test_split(X,y,test_size=0.2)
m=LogisticRegression(max_iter=200,multi_class="ovr");m.fit(Xt,yt)
print(accuracy_score(ye,m.predict(Xe)))
```

```
0.9333333333333333
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1256: FutureWarning: 'multi_class' was deprecated
warnings.warn(
```

```
# experiment no 5: predicting new data
print(m.predict([X[0]]))
```

[0]

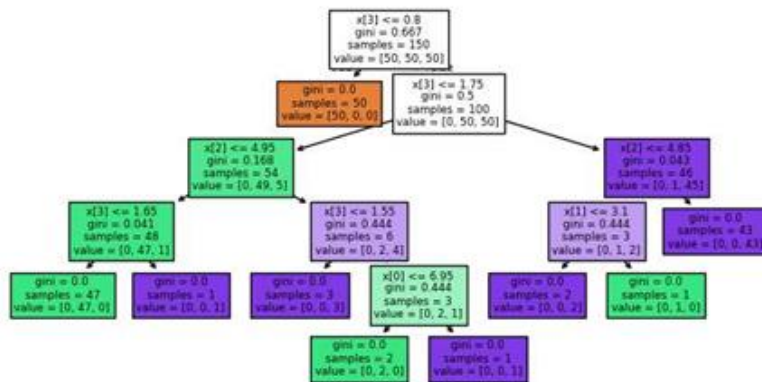
Result

Logistic regression models were successfully implemented for binary and multiclass classification. Model performance was evaluated using accuracy, confusion matrix, classification report, ROC curve, and AUC score.

Lab– 06: Decision Trees in Machine Learning

Aim: To implement and evaluate decision tree models for classification tasks using Python.

```
# experiment no 1: simple decision tree
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt
d=load_iris();X=d.data;y=d.target
m=DecisionTreeClassifier();m.fit(X,y)
plt.figure(figsize=(8,4));plot_tree(m,filled=True);plt.show()
```



```
# experiment no 2: performance evaluation
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
Xt,Xe,yt,ye=train_test_split(X,y,test_size=0.2)
m.fit(Xt,yt);yp=m.predict(Xe)
print(accuracy_score(ye,yp),precision_score(ye,yp,average="macro"),
      recall_score(ye,yp,average="macro"),f1_score(ye,yp,average="macro"))
```

1.0 1.0 1.0 1.0

```
# experiment no 3: gini vs entropy
m1=DecisionTreeClassifier(criterion="gini")
m2=DecisionTreeClassifier(criterion="entropy")
m1.fit(Xt,yt);m2.fit(Xt,yt)
print(accuracy_score(ye,m1.predict(Xe)),accuracy_score(ye,m2.predict(Xe)))
```

1.0 1.0

```
# experiment no 4: pruning and overfitting control
m=DecisionTreeClassifier(max_depth=3,min_samples_split=4,min_samples_leaf=2)
m.fit(Xt,yt)
print(accuracy_score(ye,m.predict(Xe)))
```

1.0

```
# experiment no 5: predicting new data
print(m.predict([X[0]]))
```

[0]

Result

Decision tree models were successfully implemented and evaluated. Pruning helped reduce overfitting while maintaining good accuracy.

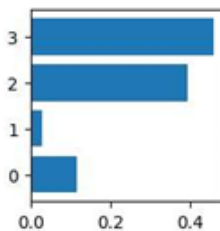
Lab– 07: Random Forests and Ensemble Methods in Machine Learning

Aim: To implement and evaluate ensemble learning techniques using Random Forests for classification and regression.

```
# experiment no 1: random forest classifier
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
d=load_iris();X=d.data;y=d.target
Xt,Xe,yt,ye=train_test_split(X,y,test_size=0.2)
rf=RandomForestClassifier(n_estimators=100);rf.fit(Xt,yt)
print(accuracy_score(ye,rf.predict(Xe)))
```

0.9666666666666667

```
# experiment no 2: feature importance
import matplotlib.pyplot as plt
imp=rf.feature_importances_
plt.figure(figsize=(2,2));plt.barh(range(len(imp)),imp);plt.show()
```



```
# experiment no 3: hyperparameter tuning
rf2=RandomForestClassifier(n_estimators=200,max_depth=5,min_samples_split=4,min_samples_leaf=2)
rf2.fit(Xt,yt)
print(accuracy_score(ye,rf2.predict(Xe)))
```

0.9666666666666667

```
# experiment no 4: random forest regression
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error,r2_score
import numpy as np
Xr=X;yr=X[:,0]
Xt,Xe,yt,ye=train_test_split(Xr,yr,test_size=0.2)
rfr=RandomForestRegressor(n_estimators=100);rfr.fit(Xt,yt)
yp=rfr.predict(Xe)
print(mean_squared_error(ye,yp),np.sqrt(mean_squared_error(ye,yp)),r2_score(ye,yp))
```

0.008021866666666728 0.08956487406716278 0.9879247353192058

```
# experiment no 5: decision tree vs random forest
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import r2_score
dt=DecisionTreeRegressor();dt.fit(Xt,yt)
rf=RandomForestRegressor();rf.fit(Xt,yt)
print(f"DT R2:{r2_score(ye,dt.predict(Xe)):.2f} RF R2:{r2_score(ye,rf.predict(Xe)):.2f}")
```

DT R2:0.99 RF R2:0.99

Result

Random Forest models were successfully implemented for classification and regression. Ensemble learning improved model performance compared to a single decision tree.

Lab-08: Support Vector Machine (SVM) in Machine Learning

Aim: To implement and evaluate SVM models for classification tasks.

```
# experiment no 1: binary svm (linear)
from sklearn.datasets import load_iris
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
d=load_iris();X=d.data;y=(d.target!=2).astype(int)
```

```
Xt,Xe,yt,ye=train_test_split(X,y,test_size=0.2)
m=SVC(kernel="linear");m.fit(Xt,yt)
print(accuracy_score(ye,m.predict(Xe)))
```

0.9

```
# experiment no 2: different kernels
for k in ["linear","poly","rbf"]:
    m=SVC(kernel=k);m.fit(Xt,yt)
    print(k,accuracy_score(ye,m.predict(Xe)))
```

```
linear 0.9
poly 0.8666666666666667
rbf 0.9
```

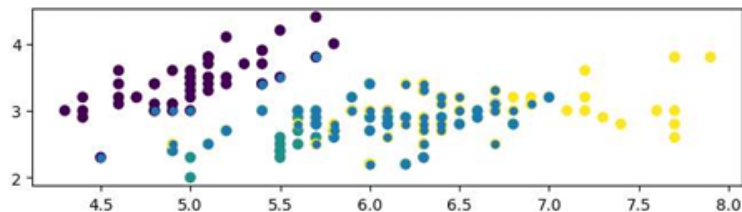
```
# experiment no 3: hyperparameter tuning
m=SVC(kernel="rbf",C=2,gamma=0.5);m.fit(Xt,yt)
print(accuracy_score(ye,m.predict(Xe)))
```

0.9

```
# experiment no 4: multiclass svm
y=d.target
Xt,Xe,yt,ye=train_test_split(X,y,test_size=0.2)
m=SVC(decision_function_shape="ovr");m.fit(Xt,yt)
print(accuracy_score(ye,m.predict(Xe)))
```

0.9333333333333333

```
# experiment no 5: decision boundary
import matplotlib.pyplot as plt
X2=X[:,0:2];y2=y
m=SVC(kernel="linear");m.fit(X2,y2)
plt.figure(figsize=(8,2))
plt.scatter(X2[:,0],X2[:,1],c=y2)
plt.scatter(m.support_vectors_[0],m.support_vectors_[1],s=20)
```



Result

Support Vector Machine models were successfully implemented for classification tasks, and model performance was evaluated using accuracy and classification metrics.

Lab– 09: K-Nearest Neighbors (KNN) in Machine Learning

Aim: To implement and evaluate KNN models for classification and regression.

```
# experiment no 1: knn classification
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, mean_squared_error, r2_score
import matplotlib.pyplot as plt, numpy as np
d=load_iris(); X=d.data; y=d.target
Xt, Xe, yt, ye=train_test_split(X, y, test_size=0.2)
m=KNeighborsClassifier(n_neighbors=5); m.fit(Xt, yt)
print(accuracy_score(ye, m.predict(Xe)))
```

0.9333333333333333

```
# experiment no 2: different k values
for k in [3, 5, 7]:
    m=KNeighborsClassifier(n_neighbors=k); m.fit(Xt, yt)
    print(k, accuracy_score(ye, m.predict(Xe)))
```

3 0.9333333333333333
5 0.9333333333333333
7 0.9666666666666667

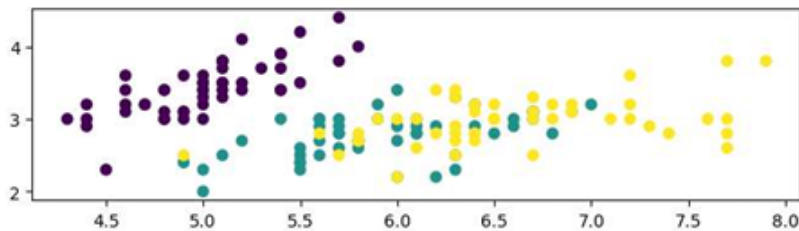
```
# experiment no 3: distance metrics
for mtr in ["euclidean", "manhattan", "minkowski"]:
    m=KNeighborsClassifier(metric=mtr); m.fit(Xt, yt)
    print(mtr, accuracy_score(ye, m.predict(Xe)))
```

euclidean 0.9333333333333333
manhattan 0.9333333333333333
minkowski 0.9333333333333333

```
# experiment no 4: knn regression
Xr=X; yr=X[:,0]
Xt, Xe, yt, ye=train_test_split(Xr, yr, test_size=0.2)
m=KNeighborsRegressor(n_neighbors=5); m.fit(Xt, yt)
yp=m.predict(Xe)
print(mean_squared_error(ye, yp), np.sqrt(mean_squared_error(ye, yp)), r2_score(ye, yp))
```

0.025506666666666615 0.15970806700560436 0.9546333076421415

```
# experiment no 5: knn decision boundary
X2=X[:,0:2]; y2=y
m=KNeighborsClassifier(n_neighbors=5); m.fit(X2, y2)
plt.figure(figsize=(8, 2))
plt.scatter(X2[:,0], X2[:,1], c=y2)
plt.show()
```



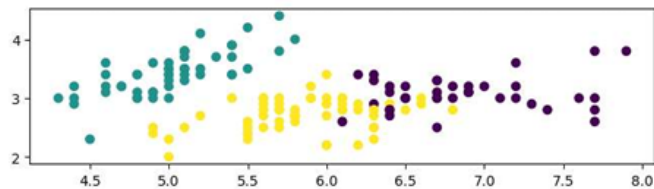
Result

K-Nearest Neighbors models were successfully implemented for both classification and regression, and model performance was evaluated using appropriate accuracy and error metrics.

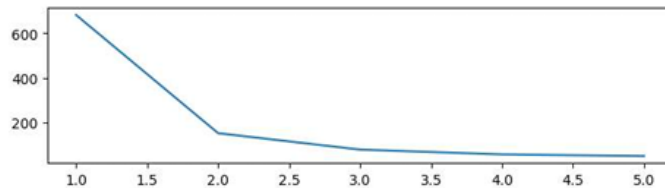
Lab– 10: Unsupervised Learning– Clustering in Machine Learning

Aim: To implement and evaluate clustering algorithms.

```
# experiment no 1: k-means clustering
from sklearn.datasets import load_iris
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
d=load_iris();X=d.data
k=KMeans(n_clusters=3);lbl=k.fit_predict(X)
plt.figure(figsize=(8,2));plt.scatter(X[:,0],X[:,1],c=lbl);plt.show()
```

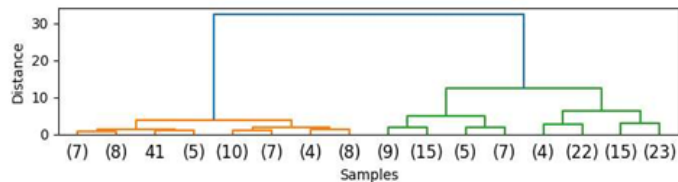


```
# experiment no 2: elbow method
wcss=[]
for i in range(1,6):
    k=KMeans(n_clusters=i);k.fit(X);wcss.append(k.inertia_)
plt.figure(figsize=(8,2));plt.plot(range(1,6),wcss);plt.show()
```

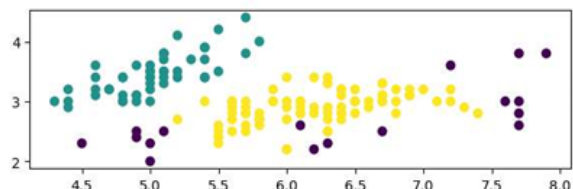


```
# experiment no 3: hierarchical clustering
from scipy.cluster.hierarchy import dendrogram,linkage
import matplotlib.pyplot as plt
Z=linkage(X,method="ward")
```

```
plt.figure(figsize=(7,2));dendrogram(Z, truncate_mode='level', p=3)
plt.xlabel("Samples");plt.ylabel("Distance");plt.tight_layout();plt.show()
```



```
# experiment no 4: dbscan clustering
from sklearn.cluster import DBSCAN
d=DBSCAN(eps=0.5,min_samples=5);lbl=d.fit_predict(X);plt.figure(figsize=(7,2));plt.scatter(X[:,0],X[:,1],c=lbl);plt.show()
```



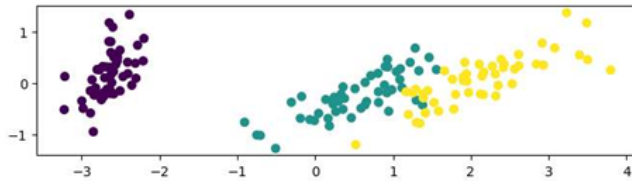
```
# experiment no 5: clustering evaluation
from sklearn.metrics import silhouette_score,davies_bouldin_score;print(silhouette_score(X,lbl),davies_bouldin_score(X,lbl))
```

```
0.48603419703456857 7.222448016359581
```

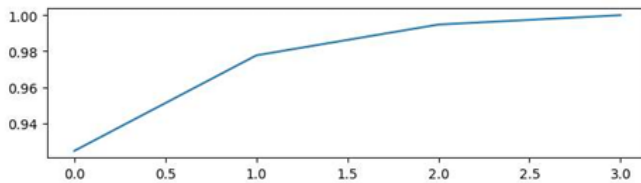
Lab– 11: Dimensionality Reduction in Machine Learning

Aim: To reduce feature space while retaining important information.

```
# experiment no 1: pca dimensionality reduction
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
d=load_iris();X=d.data;y=d.target
p=PCA(n_components=2);Xp=p.fit_transform(X)
plt.figure(figsize=(8,2));plt.scatter(Xp[:,0],Xp[:,1],c=y);plt.show()
```



```
# experiment no 2: variance explained
p=PCA();p.fit(X)
plt.figure(figsize=(8,2));plt.plot(p.explained_variance_ratio_.cumsum());plt.show()
```

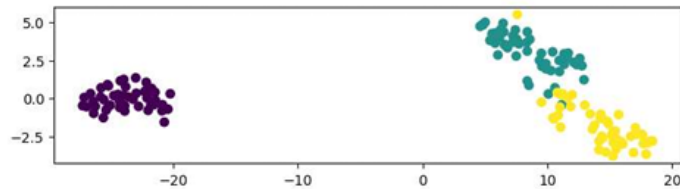


```
# experiment no 3: pca for classification
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
p=PCA(n_components=2);Xp=p.fit_transform(X)
Xt,Xe,yt,ye=train_test_split(Xp,y,test_size=0.2)

m=LogisticRegression(max_iter=200);m.fit(Xt,yt)
print(accuracy_score(ye,m.predict(Xe)))
```

1.0

```
# experiment no 4: tsne visualization
from sklearn.manifold import TSNE
t=TSNE(n_components=2,perplexity=30)
Xt=t.fit_transform(X)
plt.figure(figsize=(8,2));plt.scatter(Xt[:,0],Xt[:,1],c=y);plt.show()
```



```
# experiment no 5: pca vs tsne comparison
print(Xp.shape,Xt.shape)
```

(150, 2) (150, 2)

Result

Dimensionality reduction techniques were successfully applied to reduce the feature space while preserving important information, and model performance was effectively maintained after reduction.

Lab – 12: Model Evaluation and Cross-Validation

Aim: To evaluate machine learning models using various metrics and validate model performance using cross-validation.

```
# experiment no 1: train-test evaluation
from sklearn.datasets import load_iris
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
d=load_iris();X=d.data;y=d.target
Xt,Xe,yt,ye=train_test_split(X,y,test_size=0.2)
m=LogisticRegression(max_iter=200);m.fit(Xt,yt);yp=m.predict(Xe)
print(accuracy_score(ye,yp),precision_score(ye,yp,average="macro"),
      recall_score(ye,yp,average="macro"),f1_score(ye,yp,average="macro"))
```

```
0.9333333333333333 0.9487179487179488 0.9444444444444445 0.9419191919191919
```

```
# experiment no 2: k-fold cross validation
from sklearn.model_selection import KFold,cross_val_score
kf=KFold(n_splits=5)
print(cross_val_score(m,X,y,cv=kf).mean())
```

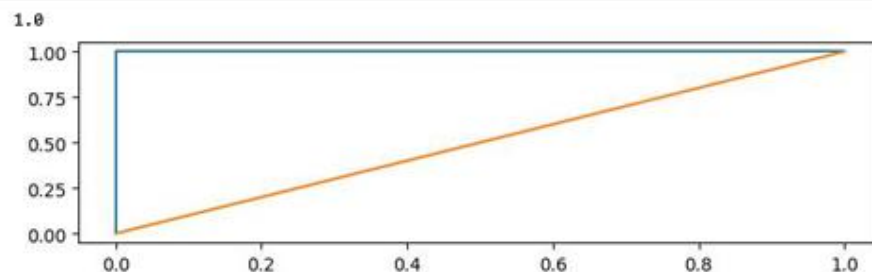
```
0.9266666666666665
```

```
# experiment no 3: stratified k-fold
from sklearn.model_selection import StratifiedKFold
skf=StratifiedKFold(n_splits=5)
print(cross_val_score(m,X,y,cv=skf).mean())
```

```
0.9733333333333334
```

```
# experiment no 4: grid search cv
from sklearn.model_selection import GridSearchCV;from sklearn.svm import SVC
g=GridSearchCV(SVC(),{"C":[1,2],"kernel":["linear","rbf"]},cv=3);g.fit(X,y);print(g.best_params_,g.best_score_)
{'C': 1, 'kernel': 'linear'} 0.9933333333333333
```

```
# experiment no 5: roc curve & auc
from sklearn.metrics import roc_curve,auc;import matplotlib.pyplot as plt
ybin=(y==0).astype(int);Xt,Xe,yt,ye=train_test_split(X,ybin,test_size=0.2)
m=LogisticRegression(max_iter=200);m.fit(Xt,yt);ys=m.predict_proba(Xe)[:,-1]
fpr,tpr,_=roc_curve(ye,ys);print(auc(fpr,tpr));plt.figure(figsize=(8,2));plt.plot(fpr,tpr);plt.plot([0,1],[0,1]);plt.show()
```



Result

Model evaluation and cross-validation techniques were successfully implemented. Model performance was assessed using train-test split, k-fold and stratified k-fold cross-validation. Hyperparameters were optimized using grid search, and model effectiveness was validated using ROC curve and AUC score.

